

ZERO → HERO

Kubernetes

The Complete Architecture & Design Guide

Every component explained in plain language — with real-world analogies, full diagrams, and a command cheat-sheet. Built for a total beginner, useful for a pro.

Architecture

Control Plane

Pods & Deployments

Networking

kubectl Cheat Sheet

Learning Roadmap

K8s = K + (8 letters) + s — "Kubernetes"

A complete beginner-to-advanced reference

What's Inside

Read it top to bottom to learn from scratch, or jump to any section as a reference later.

01	The Big Picture — What is Kubernetes, really?	03
02	Containers vs Virtual Machines	05
03	The Architecture at a Glance (Master Diagram)	06
04	The Control Plane — The Brain	08
05	Worker Nodes — The Muscle	10
06	The Building Blocks (Kubernetes Objects)	11
07	Networking Explained Simply	15
08	Storage: Volumes, PV & PVC	16
09	Life of a Request: How a Pod Gets Created	17
10	Anatomy of a YAML File	18
11	The Complete kubectl Cheat Sheet	19
12	Your Learning Roadmap (Start → End)	21
13	Glossary of Terms	22

? How to read this guide

Whenever you see a **blue analogy box**, that's the idea explained with an everyday comparison — no tech knowledge needed. **Green boxes** are handy tips, **amber boxes** are common mistakes to avoid, and dark boxes are real commands you can type.

The Big Picture — What is Kubernetes, Really?

Before any diagrams or commands, let's understand the *idea* in plain words.

Modern apps (a website, a mobile-app backend, an online shop) are packaged into little self-contained boxes called **containers**. One container might run your website, another runs your database, another handles payments. On a small scale you can start these by hand.

But real apps need **hundreds or thousands** of these containers running across many computers — and they must stay alive 24/7, survive crashes, handle traffic spikes, and update without downtime. Doing that by hand is impossible. **Kubernetes is the software that manages all of this for you, automatically.**

A Analogy: The Shipping Port

Think of a giant shipping port. **Containers** are the metal shipping boxes — each holds one type of cargo (your app). **Kubernetes is the port's control tower + crane operators.** It decides which ship (computer) each container goes on, replaces any box that falls into the water, adds more boxes when demand rises, and reroutes everything if a crane breaks — all without a human running around the docks. That automatic management is called **orchestration**.

What does Kubernetes actually do for you?

The job	What it means in plain English
Scheduling	Decides <i>which computer</i> should run each container, based on free memory/CPU.
Self-healing	If a container crashes or a whole computer dies, it restarts/replaces it automatically.
Scaling	Traffic doubled? It launches more copies. Traffic dropped? It removes the extras.
Load balancing	Spreads incoming users evenly across all the copies so none gets overwhelmed.
Rolling updates	Ships a new version gradually with <i>zero downtime</i> , and rolls back if it breaks.
Service discovery	Lets containers find and talk to each other by name, even as they move around.
Config & secrets	Injects settings and passwords into apps without baking them into the code.

i Where does the name come from?

"Kubernetes" is Greek for **helmsman** — the person who steers a ship. The nickname **K8s** is just a shorthand: the letter **K**, then **8** letters (*u-b-e-r-n-e-t-e*), then **s**. It was created at Google and is now open-source, maintained by the community (CNCF).

The one mindset shift: declarative, not step-by-step

With Kubernetes you **don't** give it a list of steps ("start this, then that"). Instead you describe the **desired end state** — e.g. "I always want 3 copies of my website running." Kubernetes constantly compares reality to your wish and fixes any difference. This is called **declarative** management.

A Analogy: The Thermostat

You don't tell a thermostat "turn the heater on for 10 minutes." You just set it to **22°C** and it does whatever it takes to keep the room there — heating when cold, resting when warm. Kubernetes is a thermostat for your apps: you set "3 copies," and it keeps exactly 3 alive forever.

Containers vs Virtual Machines

Kubernetes runs **containers**. To understand why they’re special, compare them to the older approach: virtual machines.

Virtual Machine (VM) — the old way

A VM is a whole **fake computer** running inside your real computer, including its *own full operating system*. It’s powerful but heavy: gigabytes in size and slow (minutes) to start.

Container — the modern way

A container packages **only your app + its needs**, and shares the host’s operating system. It’s tiny (megabytes) and starts in *seconds*. You can run many more on the same hardware.

A Analogy: Houses vs Apartments

A **VM is a detached house** — it has its own foundation, plumbing, and walls (a full OS). Great isolation, but expensive and space-hungry. A **container is an apartment** in a shared building — it has its own private rooms (your app) but shares the building’s foundation and plumbing (the host OS). You fit far more apartments than houses on the same plot of land.

Side-by-side

Feature	Virtual Machine	Container
Size	Gigabytes (whole OS)	Megabytes (just the app)
Startup time	Minutes	Seconds or less
Isolation	Very strong (separate OS)	Strong (shared OS kernel)
How many per server	A handful (dozens)	Hundreds
Best for	Running different operating systems	Packaging & scaling apps quickly

Key words you now know

Term	Plain meaning
Image	A frozen, ready-to-run <i>template</i> of your app (like a recipe or an installer).
Container	A <i>running copy</i> of an image (the recipe actually being cooked).
Docker	The most popular tool for building images and running containers.
Registry	An online store/library of images (e.g. Docker Hub) you pull from.
Orchestration	Automatically managing <i>many</i> containers — exactly Kubernetes’ job.

✓ The relationship in one line

Docker builds and runs one container. **Kubernetes** manages thousands of them across many machines. They’re teammates, not competitors.

Common beginner confusion

Containers are **not** “lightweight VMs.” A VM virtualizes *hardware* (it fakes a whole computer). A container virtualizes the *operating system* (it just isolates a process). Different technique, different purpose.

The Architecture at a Glance

Everything in Kubernetes lives inside a **Cluster**. A cluster has two kinds of machines: one **brain** (the Control Plane) and one or more **workers** (Nodes) that run your apps.

A Analogy: A Restaurant Kitchen

The **Control Plane** is the **head chef + manager** — they take orders, plan, and assign work but don't cook. The **Worker Nodes** are the **line cooks** — they actually make the food (run your apps). You (the developer) are the **customer** handing your order to the manager via `kubectl`.

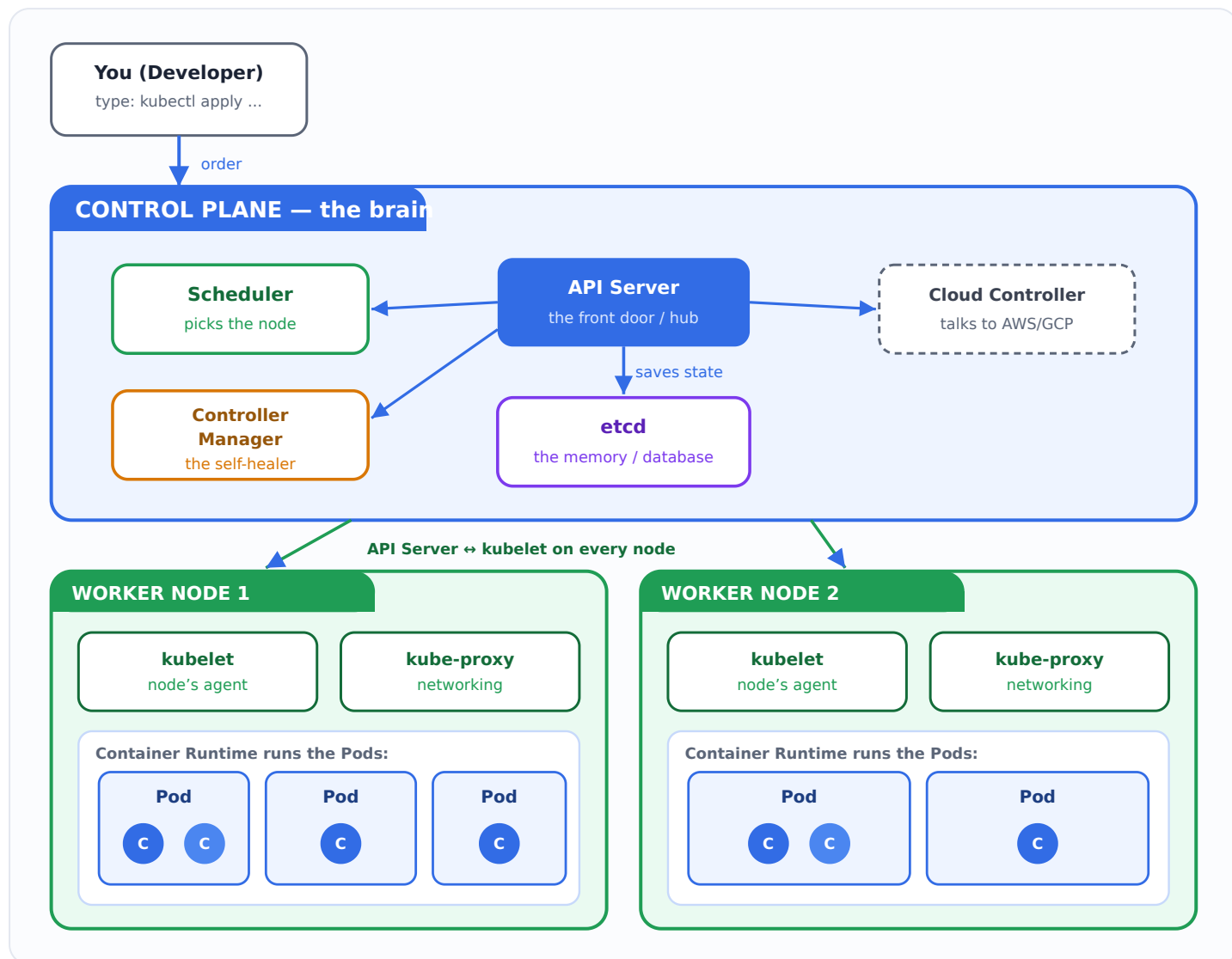


Fig 1. — The full cluster. **C** = a container inside a Pod. The API Server is the hub every part talks through.

Reading the Map

The diagram has three layers. Here's the whole thing in one breath, then we'll zoom into each part.

- 1 **You** describe what you want ("run 3 copies of my app") and send it with the `kubectl` command.
- 2 The request hits the **API Server** — the single front door to the whole cluster. Everything goes through it.
- 3 Your wish is saved into **etcd**, the cluster's memory/database. This is now the "desired state."
- 4 The **Scheduler** notices new work with no home and picks the best **Worker Node** for it.
- 5 The **kubelet** on that node starts the containers using the **container runtime**.
- 6 The **Controller Manager** keeps watching forever — if a copy dies, it asks for a replacement so reality matches your wish.

The two halves, summarized

Layer	Contains	Its job in one sentence
Control Plane (the brain)	API Server, etcd, Scheduler, Controller Manager, Cloud Controller	Makes all the decisions and stores the cluster's state — but runs none of your apps.
Worker Nodes (the muscle)	kubelet, kube-proxy, container runtime, your Pods	Actually runs your application containers and does the real work.

✓ The golden rule of Kubernetes

Everything talks to the API Server. Nothing talks directly to anything else. The Scheduler, controllers, and every kubelet all read from and write to the API Server only. That's why it's drawn in the center — it's the hub of the whole wheel.

i Desired state vs current state

This is the heartbeat of Kubernetes. **Desired state** = what you asked for (stored in etcd). **Current state** = what's really running right now. Controllers run an endless loop: *"compare the two → if different, take action → repeat."* This loop is called a **reconciliation loop**, and it's why the system heals itself.

The Control Plane — The Brain

These five components make every decision. They usually run together on a dedicated machine (or several, for reliability). Let's meet each one.

1. kube-apiserver — The Front Door

The only way in or out of the cluster.

Every command, every component, every check-in passes through the API Server. It validates requests, decides who's allowed to do what, and reads/writes the cluster state. If it goes down, you can't control the cluster (though running apps keep running).

A Like a receptionist

Nobody walks into the building directly — every visitor checks in at the front desk, which verifies their ID and routes them. The API Server is that reception desk for the whole cluster.

2. etcd — The Memory

A reliable database that stores the entire cluster state.

etcd is a **key-value database** that remembers *everything*: what should be running, what config exists, what secrets are set. It is the single source of truth. If you backed up only one thing in a cluster, you'd back up etcd.

A Like the cluster's notebook

Everything the cluster knows is written in this notebook. Lose the notebook and the cluster forgets who it is. That's why etcd is protected and backed up carefully.

3. kube-scheduler — The Placement Officer

Decides which node a new Pod should run on.

When you create a Pod, it has no home yet. The Scheduler looks at every node's free CPU and memory, plus any rules you set ("keep these apart," "this needs a GPU"), and picks the best fit. It only *decides* — the kubelet does the actual starting.

A Like a restaurant host

A new party arrives; the host scans the room and seats them at the table that fits best. The host doesn't cook or serve — just assigns the seat.

4. kube-controller-manager — The Self-Healer

Runs the loops that keep reality matching your wishes.

It bundles many small **controllers**, each watching one thing. The *Node controller* notices dead nodes. The *ReplicaSet controller* makes sure the right number of Pod copies exist. The *Job controller* runs one-off tasks. Each runs the reconciliation loop: **compare** → **fix** → **repeat**.

A Like a building supervisor

They constantly patrol: a light is out? Replace it. A door's unlocked? Lock it. Always nudging the building back to how it's supposed to be.

5. cloud-controller-manager — The Cloud Translator (optional)

Connects Kubernetes to your cloud provider (AWS, Google Cloud, Azure).

When your cluster runs in the cloud, this component asks the provider for real load balancers, storage disks, and network routes on Kubernetes' behalf. On a laptop or private server, you won't have this at all.

Worker Nodes — The Muscle

Nodes are the machines (physical or virtual) that actually run your app containers. Every node runs three things.

1. kubelet — The Node's Agent

The captain on each node that takes orders from the Control Plane.

The kubelet is the one part of the worker that talks to the API Server. It receives “run these Pods,” tells the container runtime to start them, then continuously reports back: “Pod healthy, Pod crashed, node has this much memory left.” If a Pod dies, the kubelet is who notices first.

A Like a shift manager

Head office (Control Plane) sends the day's tasks; the shift manager makes sure the workers do them and phones back with status updates.

2. kube-proxy — The Traffic Router

Makes networking work so Pods can be reached by a stable address.

Pods come and go and their internal IP addresses change constantly. kube-proxy maintains the network rules on each node so that traffic sent to a **Service** (a fixed name/address) is correctly forwarded to whichever live Pods are behind it — and load-balanced across them.

A Like a call-center switchboard

You dial one number (the Service); the switchboard connects you to whichever agent (Pod) is free, without you needing to know their personal extension.

3. Container Runtime — The Engine

The software that actually starts, stops, and runs containers.

The kubelet doesn't run containers itself — it delegates to a container runtime such as **containerd** or **CRI-O**. (Docker was the classic one; modern clusters use containerd underneath.) This is the muscle that pulls the image and boots the container.

A Like the kitchen's stove

The shift manager (kubelet) says “cook this dish,” but the stove (runtime) is what applies the actual heat and produces the meal.

✓ Putting nodes together

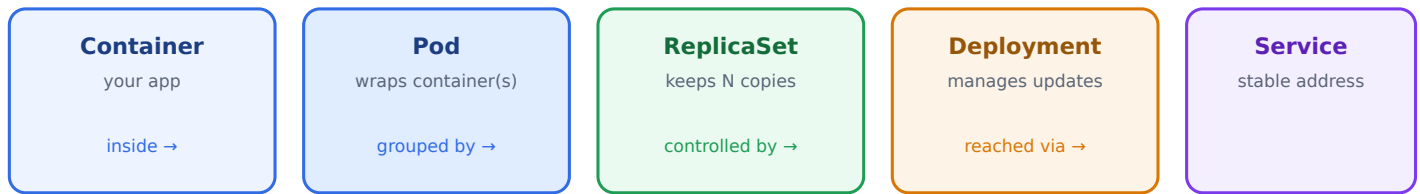
A cluster can have anywhere from 1 to thousands of worker nodes. Need more capacity? Add nodes. Kubernetes automatically starts using them — no app changes required. This is **horizontal scaling**.

! Node vs Pod — don't mix them up

A **Node** is a whole computer. A **Pod** is a small wrapper around your container(s) that *runs on* a node. One node hosts many pods. We'll define pods next.

The Building Blocks (Objects)

You control Kubernetes by creating **objects** — the nouns of the system. Here they are, layered from the smallest up. Each builds on the one before.



Pod — the smallest unit you deploy

A Pod wraps **one container** (usually) plus everything it needs to run. Containers in the same Pod share a network address and storage — they're glued together. **You rarely create Pods directly**; you let higher objects (Deployments) create them for you, because a lone Pod isn't replaced if it dies.

A Like a pea pod

A pea pod holds one or more peas (containers) that belong together and travel as a unit. That's literally where the name comes from.

ReplicaSet — the copy-keeper

Its only job: **make sure exactly N identical Pods are always running**. Asked for 3 and one dies? It immediately creates a fresh one. You almost never make these by hand either — a Deployment makes them for you.

Deployment — the one you'll use most

This is the everyday workhorse for stateless apps (websites, APIs). It manages ReplicaSets for you and adds the superpower of **rolling updates**: push a new version and it swaps Pods gradually with zero downtime, and can **roll back** instantly if something breaks.

A Like a project manager

You tell the manager the goal ("3 copies, version 2"). They handle hiring replacements, rolling out the new version worker-by-worker, and reverting if it goes wrong — you never micromanage individuals.

✓ The chain in one line

You create a **Deployment** → it creates a **ReplicaSet** → which creates **Pods** → each Pod runs your **Container**. Delete the Deployment and the whole chain is cleaned up.

Connecting & Configuring

Pods are mortal and their addresses change. These objects give apps a stable way to be found, configured, and reached from outside.

Service — the stable front for changing Pods

Pods are constantly created and destroyed, each with a new IP. A **Service** gives a permanent name and address that automatically points to whichever Pods are currently alive, and load-balances across them. There are three main types:

Type	Who can reach it	Use for
ClusterIP (default)	Only things <i>inside</i> the cluster	Internal apps talking to each other (e.g. web → database)
NodePort	Anyone who can reach a node's IP + port	Simple external access, testing, demos
LoadBalancer	The public internet	Real production traffic (asks the cloud for a load balancer)

A Like a company phone number

Employees (Pods) come and go and change desks, but the company's main number (Service) never changes and always routes your call to someone available.

Ingress — the smart front gate

An Ingress sits in front of many Services and routes web traffic by **URL or hostname** — e.g. `shop.site.com` → the shop Service, `blog.site.com` → the blog Service. It also handles HTTPS certificates. One entry point, many destinations.

A Like a building directory + receptionist

One street address (your domain); the directory reads where you want to go and sends you to the right floor (Service). Without it, every department would need its own street entrance.

ConfigMap — settings

Stores **non-secret** configuration (URLs, feature flags, tuning values) separately from your code, then injects it into Pods. Change a setting without rebuilding the app.

Secret — passwords

Like a ConfigMap but for **sensitive** data: passwords, API keys, tokens. Stored with extra care and kept out of your code and images.

Namespace — virtual folders

A way to split one physical cluster into isolated **virtual sections** — e.g. `dev`, `staging`, `prod`, or one per team. Names only need to be unique within a namespace, and you can set separate permissions and resource limits per namespace.

A Like floors in an office building

One building (cluster), but each floor (namespace) is a separate workspace with its own teams and rules, without needing separate buildings.

Other Workload Types

Deployments cover most apps, but some jobs need special behavior. Here are the other four you'll meet.

StatefulSet — for apps that remember

When each copy needs a stable identity and its own storage.

Deployments treat all copies as interchangeable and disposable. But databases can't work that way — each copy needs a **fixed name** and **its own permanent disk**. A StatefulSet gives Pods predictable names (`db-0` , `db-1`) and keeps their storage attached even after restarts.

A Named lockers vs shared bins

A Deployment's Pods share generic bins — any bin works. A StatefulSet gives each Pod a personal locker with its name on it that it gets back every time.

DaemonSet — one copy on every node

When something must run on all nodes, no more no less.

Ensures a copy of a Pod runs on **every** node in the cluster — and automatically on any new node you add. Perfect for background agents that watch each machine: log collectors, monitoring, security scanners.

A Like a smoke detector per room

You don't want 3 detectors in one room and none in another — you want exactly one in every room. Add a room, it gets one too.

Job — run once, then stop

For a task that **runs to completion** and finishes — a data import, a report, a one-off migration. Unlike a Deployment, it's not meant to run forever; it succeeds and exits.

CronJob — run on a schedule

A Job that fires **on a timer** — every night at 2 a.m., every hour, etc. Great for backups, cleanups, and recurring reports. Same idea as a scheduled task on your computer.

Quick chooser: which workload do I use?

If your app...	Use this
is a normal website/API (stateless)	Deployment
is a database needing stable identity + storage	StatefulSet
must run on every node (agent/monitor)	DaemonSet
does one task then finishes	Job
runs on a repeating schedule	CronJob

✓ Rule of thumb

When in doubt, start with a **Deployment**. Reach for the others only when you hit their specific need (permanent identity, per-node, one-shot, or scheduled).

Networking Explained Simply

Networking scares beginners, but the model is built on four clear promises.

The four rules of Kubernetes networking

- 1 **Every Pod gets its own IP address.** No fighting over ports — each Pod is like its own little computer on the network.
- 2 **Any Pod can talk to any other Pod** directly, on any node, with no special setup (in a default cluster).
- 3 **Services provide stable addresses** so you never rely on a Pod's IP, which changes when it restarts.
- 4 **DNS lets you use names, not numbers.** Reach a service by its name (e.g. `database`) and the cluster resolves it for you.

How traffic reaches your app from the internet

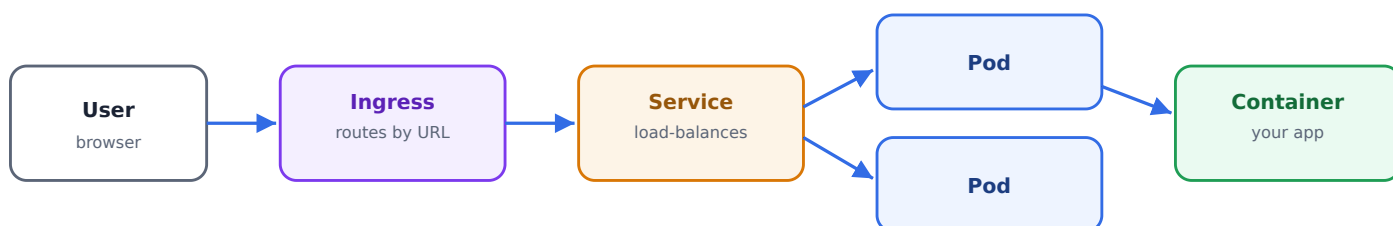


Fig 2. — The path from a user's browser to your running code.

i Who actually wires this up?

A **CNI plugin** (Container Network Interface — e.g. Calico, Flannel, Cilium) is the add-on that implements rule 1 and 2. Kubernetes defines *what* the network must do; the CNI plugin does *how*. You pick one when building a cluster.

Storage: Keeping Data Safe

Containers are disposable — when a Pod dies, anything saved inside it is **gone**. For data that must survive, Kubernetes has a storage system.

! The problem in one sentence

A container's own filesystem is **temporary**. Restart the Pod and its files reset to the original image. Databases, uploads, and logs need something that outlives the Pod.

Volume — storage attached to a Pod

A Volume is a folder that lives **outside** the container's throwaway layer, so data in it survives container restarts. Some volume types even survive the whole Pod being deleted, by storing data on a real disk elsewhere.

PersistentVolume (PV)

A piece of **real storage** in the cluster (a cloud disk, a network drive) — set up by an admin, existing independently of any Pod. Think: *"a hard drive available for rent."*

PersistentVolumeClaim (PVC)

A **request** for storage by an app: "I need 10 GB." Kubernetes matches the claim to an available PV. Think: *"the rental agreement that reserves a drive."*

A Analogy: Renting a storage unit

The **PersistentVolume** is the actual storage unit at the facility. The **PersistentVolumeClaim** is your rental contract saying "I want a 10 m² unit." Your Pod (you) then uses the unit through that contract. When the Pod moves to another node, the contract — and your stuff — move with it.

The flow

- 1 Admin creates **PersistentVolumes** (or sets up auto-creation with a *StorageClass*).
- 2 Your app files a **PersistentVolumeClaim** for the size/type it needs.
- 3 Kubernetes **binds** the claim to a matching volume and mounts it into your Pod.
- 4 Your data now **persists** even if the Pod is deleted and recreated elsewhere.

✓ StorageClass = automatic PVs

In the cloud you rarely make PVs by hand. A **StorageClass** auto-creates a real disk the moment a claim is filed. You just ask for storage and it appears.

Life of a Request

Let's trace exactly what happens the moment you ask Kubernetes to run an app. This ties every component together.

The command:

```
kubectl apply -f my-app.yaml # "please make the cluster look like this file"
```

- 1 You → API Server.** kubectl sends your YAML to the API Server. It checks *who you are* (authentication), *what you're allowed to do* (authorization), and whether the request is valid.
- 2 API Server → etcd.** The validated request ("I want a Deployment with 3 Pods") is written to etcd as the new **desired state**. Nothing is running yet — it's just recorded.
- 3 Deployment controller acts.** Watching the API Server, it sees the new Deployment and creates a **ReplicaSet**, which in turn creates 3 **Pod** objects — still just records, unscheduled, with no node.
- 4 Scheduler assigns nodes.** It spots the 3 homeless Pods, evaluates each node's free resources and rules, and writes its decision back: "Pod A → Node 1, Pod B → Node 2..."
- 5 kubelet starts them.** Each node's kubelet sees Pods assigned to it, tells the **container runtime** to pull the image and start the containers, and mounts any storage/config.
- 6 kube-proxy wires networking.** Network rules update so the app's **Service** can route traffic to the new Pods.
- 7 Reporting back.** kubelet continuously reports Pod health to the API Server, which updates etcd. Now **current state = desired state**. Done.

i And it never stops watching

If one of those Pods crashes at 3 a.m., the ReplicaSet controller notices current state (2 Pods) $\neq$ desired state (3), and the whole dance repeats to create a replacement — while you sleep. **That's self-healing in action.**

Anatomy of a YAML File

You describe objects in **YAML** — a simple text format. Almost every file has the same four top-level sections. Learn them once and every file makes sense.

DEPLOYMENT.YAML — A COMPLETE, ANNOTATED EXAMPLE

```
apiVersion: apps/v1           # WHICH API version to use
kind: Deployment              # WHAT type of object this is
metadata:                     # WHO it is (name, labels)
  name: web-app
  labels:
    app: web
spec:                          # HOW it should behave (the wish)
  replicas: 3                  # keep 3 copies alive
  selector:
    matchLabels:
      app: web                 # manage Pods with this label
  template:                    # the Pod blueprint to stamp out
    metadata:
      labels:
        app: web
    spec:
      containers:
        - name: web
          image: nginx:1.27     # which image to run
          ports:
            - containerPort: 80 # the app listens on port 80
```

The four sections, decoded

Section	Question it answers	Example
apiVersion	Which set of features/version?	apps/v1
kind	What object am I making?	Deployment, Service
metadata	What's its name & labels?	name: web-app
spec	What's the desired behavior?	replicas: 3

! YAML's one big trap: indentation

YAML uses **spaces** (never tabs) to show structure. Getting the indentation wrong is the #1 beginner error. Be consistent — 2 spaces per level is standard.

✓ Labels & selectors: the glue

Labels are sticky notes on objects (`app: web`). **Selectors** are how one object finds others by their labels. A Service uses a selector to find its Pods; a Deployment uses one to manage its Pods. Master this pairing and half of Kubernetes clicks.

The kubectl Cheat Sheet

kubectl (“kube-control”) is your remote control for the cluster. Here are the commands you’ll actually use, grouped by task. `<name>` means “put your own name here.”

Looking around (read-only — always safe)

Command	What it does
<code>kubectl get pods</code>	List all Pods
<code>kubectl get pods -o wide</code>	List Pods with node & IP info
<code>kubectl get all</code>	List Pods, Services, Deployments...
<code>kubectl get nodes</code>	List the worker machines
<code>kubectl get svc</code>	List Services
<code>kubectl get deploy</code>	List Deployments
<code>kubectl describe pod <name></code>	Full details + recent events (great for debugging)
<code>kubectl get pods -n <namespace></code>	List Pods in a specific namespace
<code>kubectl get pods -A</code>	List Pods across ALL namespaces
<code>kubectl get pods --watch</code>	Live-update the list as things change

Creating & updating

Command	What it does
<code>kubectl apply -f <file>.yaml</code>	Create/update from a YAML file (the main one)
<code>kubectl apply -f ./folder/</code>	Apply every YAML in a folder
<code>kubectl create deploy web --image=nginx</code>	Quick-create a Deployment without a file
<code>kubectl scale deploy web --replicas=5</code>	Change the number of copies to 5
<code>kubectl set image deploy/web web=nginx:1.28</code>	Roll out a new image version
<code>kubectl edit deploy web</code>	Open the live object in an editor
<code>kubectl rollout status deploy/web</code>	Watch an update progress
<code>kubectl rollout undo deploy/web</code>	Roll back to the previous version

Deleting

Command	What it does
<code>kubectl delete pod <name></code>	Delete a Pod (it may be recreated by its owner)
<code>kubectl delete -f <file>.yaml</code>	Delete everything defined in a file
<code>kubectl delete deploy web</code>	Delete a Deployment and all its Pods

Cheat Sheet — Debugging & Pro Tips

Debugging (your daily lifesavers)

Command	What it does
<code>kubectl logs <pod></code>	Show a Pod's output/logs
<code>kubectl logs <pod> -f</code>	Stream (follow) logs live
<code>kubectl logs <pod> --previous</code>	Logs from the crashed previous run
<code>kubectl exec -it <pod> -- bash</code>	Open a shell <i>inside</i> a running container
<code>kubectl exec <pod> -- ls /app</code>	Run a single command inside a Pod
<code>kubectl describe pod <pod></code>	See events — why a Pod won't start
<code>kubectl get events --sort-by=.lastTimestamp</code>	Recent cluster events, newest last
<code>kubectl port-forward <pod> 8080:80</code>	Tunnel a Pod's port to your laptop
<code>kubectl top pod</code>	Live CPU/memory use per Pod
<code>kubectl top node</code>	Live CPU/memory use per node

Context, namespaces & config

Command	What it does
<code>kubectl config get-contexts</code>	List clusters you can connect to
<code>kubectl config use-context <name></code>	Switch to another cluster
<code>kubectl create namespace <name></code>	Make a new namespace
<code>kubectl config set-context --current --namespace=dev</code>	Default to a namespace so you can drop <code>-n</code>

Explain & discover (learn without leaving the terminal)

Command	What it does
<code>kubectl explain pod.spec</code>	Built-in docs for any field
<code>kubectl api-resources</code>	List every object type & its short name
<code>kubectl get pod <pod> -o yaml</code>	Dump an object's full YAML

✓ Pro shortcuts that save hours

- **Aliases:** most pros set `alias k=kubectl` — then type `k get pods`.
- **Short names:** `po` =pods, `svc` =services, `deploy` =deployments, `ns` =namespaces, `no` =nodes.
- **Dry run:** add `--dry-run=client -o yaml` to *generate* a YAML file instead of applying — a great way to scaffold files.
- **Autocomplete:** enable shell completion so `Tab` finishes commands and Pod names.

Think before you delete

`kubectl delete` acts immediately with no undo prompt. Double-check the **namespace** and **context** you're in — deleting in `prod` when you meant `dev` is a classic painful mistake.

Your Learning Roadmap

A realistic path from total beginner to confident user. Don't rush — do the hands-on step at each stage.

Stage 0 Foundations (before Kubernetes)

- Learn the **Linux command line** basics: files, permissions, editing text.
- Understand **containers & Docker**: build an image, run a container, push to a registry.
- Get comfortable reading **YAML**.

Stage 1 Get a cluster & run something

- Install a local cluster: **Minikube**, **kind**, or Docker Desktop's built-in Kubernetes.
- Install `kubectl` and run `kubectl get nodes`.
- Deploy your first app with a **Deployment** + **Service**, then open it in a browser.

Stage 2 Core objects & daily work

- Master **Pods, Deployments, Services, Namespaces, Labels/Selectors**.
- Practice **scaling, rolling updates, and rollbacks**.
- Get fluent in debugging: `logs`, `describe`, `exec`, `port-forward`.
- Add **ConfigMaps** and **Secrets** to an app.

Stage 3 Real-world topics

- **Storage**: PersistentVolumes, PVCs, StatefulSets for databases.
- **Networking**: Ingress, DNS, network policies.
- **Health & limits**: liveness/readiness probes, resource requests & limits.
- **Autoscaling** (HPA) and **DaemonSets/Jobs/CronJobs**.

Stage 4 Production & ecosystem

- **Helm** (package manager for Kubernetes) to install/share apps.
- **Monitoring** (Prometheus + Grafana) and centralized **logging**.
- **RBAC** & security, **GitOps** (Argo CD / Flux), and CI/CD pipelines.
- A managed cloud cluster: **EKS** (AWS), **GKE** (Google), or **AKS** (Azure).

✓ The single best habit

Break things on purpose in your local cluster and fix them: delete a Pod and watch it heal, push a broken image and roll it back, scale to zero and back up. Hands-on beats reading every time.

Glossary of Terms

Every important word in this guide, in one place, in plain English.

Term	Plain-English meaning
Cluster	The whole system: a control plane plus worker nodes, acting as one.
Node	A single machine (real or virtual) in the cluster.
Control Plane	The brain — makes decisions, stores state, runs no apps itself.
API Server	The single front door; everything talks through it.
etcd	The database storing the entire cluster state.
Scheduler	Decides which node a new Pod runs on.
Controller	A loop that keeps reality matching your desired state.
kubelet	The agent on each node that starts and watches Pods.
kube-proxy	Handles per-node networking so Services work.
Container runtime	The engine (e.g. containerd) that actually runs containers.
Container	A running, isolated package of your app + its dependencies.
Image	The frozen template a container is created from.
Pod	The smallest deployable unit; wraps one or more containers.
ReplicaSet	Keeps a set number of identical Pods running.
Deployment	Manages ReplicaSets; handles updates & rollbacks for stateless apps.
StatefulSet	Like a Deployment but for apps needing stable identity + storage.
DaemonSet	Runs exactly one Pod on every node.
Job / CronJob	Run-once task / run-on-a-schedule task.
Service	A stable name/address in front of a changing set of Pods.
Ingress	Routes external web traffic to Services by URL/host.
ConfigMap	Non-secret configuration injected into Pods.
Secret	Sensitive data (passwords, keys) handled with extra care.
Namespace	A virtual sub-division of a cluster for isolation.
Label / Selector	Sticky-note tags on objects / the way objects find each other by tag.
Volume / PV / PVC	Storage for a Pod / a real storage resource / a request for storage.
Desired vs Current state	What you asked for vs what's really running; controllers close the gap.
kubectl	The command-line remote control for the cluster.
Helm	A package manager for installing apps into Kubernetes.
CNI	The plugin that implements cluster networking (Calico, Flannel...).



You now understand Kubernetes.

You know the brain and the muscle, the objects you deploy, how a request flows end-to-end, how networking and storage work, and the commands to drive it all. Keep this guide beside you and, most importantly — **go build something.**

Control Plane ✓

Nodes ✓

Pods & Deployments ✓

Networking ✓

Storage ✓

kubectl ✓

The best next command to run:

```
kubectl get nodes
```